

망고아이 3인 공동개발 가이드

GitHub · 바탕화면 폴더 · Claude 를 셋이 같이 쓰는 법
고친 것을 합치고 → 검사하고 → 배포해서 → test.mangoi.co.kr 에서 확인하기까지

저장소 : github.com/navy111p-sudo/mangoiweb (비공개)

실서비스 주소 : <https://test.mangoi.co.kr>

대상 : 사장님 1명 + 직원 2명 (총 3명)

목차

1. 먼저 이 한 장 — 전체 그림
2. 바탕화면의 망고아이 폴더들 정제 (지금 6개나 있습니다)
3. 처음 한 번만 하는 준비 (직원용)
4. 누가 어떤 파일을 맡는가 — 충돌을 없애는 유일한 방법
5. 매일 하는 4단계
6. 합치기(merge) — 사장님이 검사하는 법
7. 검사 3종 — 무엇으로 확인했는지 말할 수 있어야 합니다
8. 배포 — 길이 두 개 있고, 서로를 모릅니다
9. test.mangoi.co.kr 에서 실제로 확인하기
10. 사고가 났을 때 (되돌리기)
11. Claude 를 셋이 같이 쓰는 법
12. 절대 하지 말 것 · 용어사전 · 요약 카드

1. 먼저 이 한 장 — 전체 그림

1-1. 한 문장 요약

인터넷에 "진짜 원본" 폴더를 하나 두고, 세 사람이 각자 복사본을 받아 고친 뒤, 원본에 다시 합칩니다. 합쳐진 원본이 실제 홈페이지로 나갑니다.

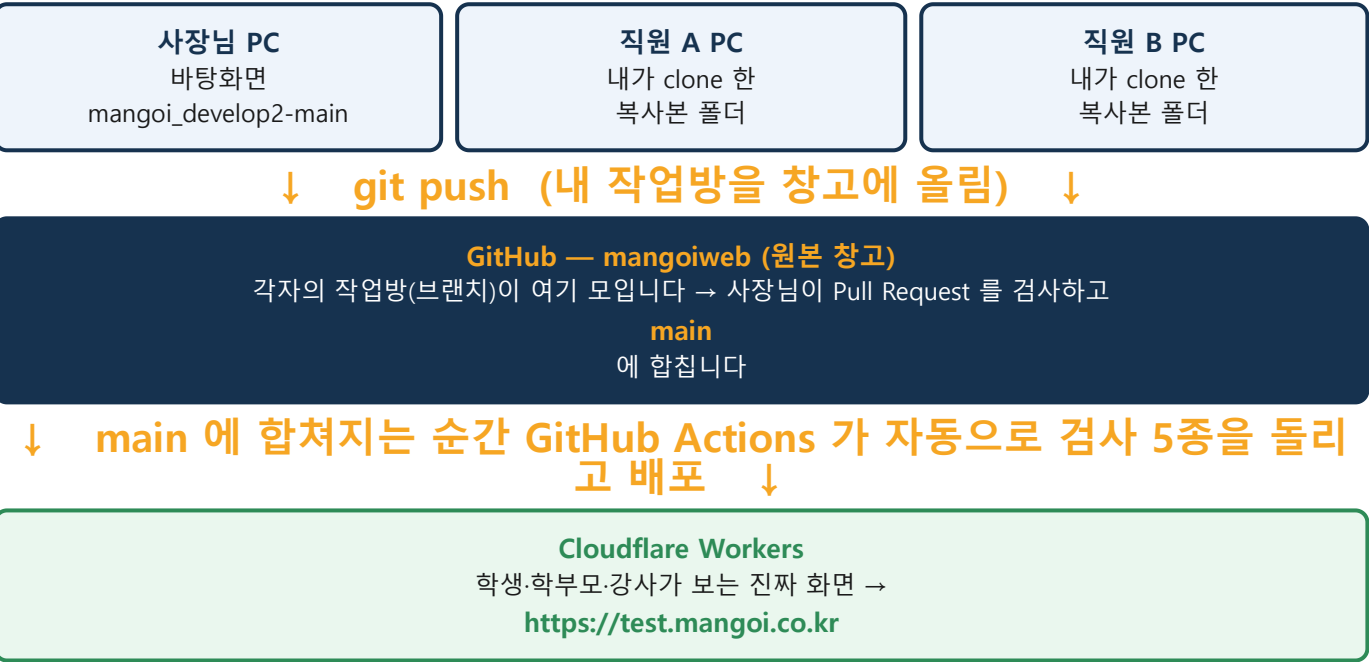
- 그 방법의 이름 = 깃(Git)

- 그 인터넷 창고의 이름 = 깃허브(GitHub)
- 홈페이지가 실제로 돌아가는 곳 = Cloudflare (주소는 `test.mangoi.co.kr`)

가장 중요한 오해 하나

바탕화면에 있는 망고아이 폴더는 **원본이 아닙니다**. 복사본입니다. 내 폴더에서 아무리 파일을 고쳐도, **홈페이지는 1도 안 바뀝니다**. 홈페이지가 바뀌려면 "합치기 → 배포" 두 단계를 더 거쳐야 합니다. (6장·8장)

1-2. 그림으로 보면



※ 직원 2명이 **같은 게임 파일**을 동시에 고치면 또 충돌합니다. A·B 사이에서도 파일을 나눠야 합니다 (4장).

2. 바탕화면의 망고아이 폴더들 정체

사장님 PC 바탕화면에는 지금 망고아이 폴더가 **여러 개** 있습니다. 헷갈리기 쉬우니 정리합니다.

폴더	정체	설명
mangoi_develop2-main	본체	진짜 작업 폴더입니다. 여기가 원본 참고와 연결돼 있습니다. 평소엔 여기서만 작업하세요.
mangoi-sec-wt mangoi_melca_deploy_wt mangoi_qa0807_deploy_wt mangoi_wt_audit	분신 (worktree)	본체와 같은 저장소를 공유하는 별도 작업방 입니다. 배포·점검할 때 "깨끗한 상태"가 필요해서 만든 것입니다. 겉보기엔 똑같은 폴더로 보이지만 서로 다른 시점의 코드 가 들어 있습니다.
mangoi_deploy_wt_260808 등	잔재	이미 등록이 풀린 옛 폴더입니다. 여기서 작업하면 아무 데도 반영되지 않습니다.

직원분들이 절대 헷갈리면 안 되는 것

직원 PC 에는 폴더가 **딱 1개만 있어야 합니다**. 3장에서 `git clone` 으로 만드는 그 폴더 하나입니다. 분신(worktree)은 배포·점검용 고급 기능이라 직원은 만들 필요도, 만들어서도 안 됩니다.

그리고 **같은 폴더를 카톡·USB·구글드라이브로 주고받지 마세요**. 그 순간 깃이 하는 일이 전부 무의미해 집니다.

2-1. 내 폴더가 지금 어떤 상태인지 확인하는 법

폴더에서 아래 두 줄만 치면, 내가 지금 **어느 작업방**에 있고 **안 올린 게 뭔지** 다 나옵니다.

```
# 지금 내가 있는 작업방(브랜치) 이름
git branch --show-current

# 안 올린 변경 파일 목록
git status
```

참고로 **지금 사장님 본체 폴더**는 `main` 이 아니라 `vc-device-helper-260810` 이라는 작업방에 들어가 있고, 아직 올리지 않은 파일이 여러 개 있는 상태입니다. 이런 상태에서 배포하면 **그 미완성 파일들이 같이 올라갑니다**.

3. 처음 한 번만 하는 준비 (직원용)

1

GitHub 계정 만들기

github.com 에서 무료 가입합니다. 사장님께
가입한 아이디
를 알려주세요.

2

사장님이 초대 → 직원이 수락

저장소가

비공개(private)

라, 초대받지 않으면 주소를 알아도 열리지 않습니다. 사장님이 저장소 → Settings → Collaborators 에서 아이디를 추가하면, 직원 메일로 초대장이 옵니다.

수락 버튼을 눌러야

접근됩니다.

(현재 등록된 사람은 사장님 1명뿐입니다. 직원 2명 초대가 첫 할 일입니다.)

3

Git 설치

git-scm.com 에서 받아 설치합니다. 옵션은 전부 기본값으로 두면 됩니다.

4

내 컴퓨터로 복사본 받기 (clone)

바탕화면에서 마우스 오른쪽 → "터미널에서 열기" 후 아래를 붙여넣습니다.

```
git clone --depth 50 https://github.com/navy111p-sudo/mangoiweb.git
```

--depth 50 = "최근 기록 50개만 받기". 전부 받으면 1.3GB라 오래 걸립니다.

5

내 이름 등록 (누가 고쳤는지 기록되게)

```
git config --global user.name "홍길동"
git config --global user.email "본인메일@example.com"
```

6

Claude Code 설치

설치 후 이 폴더에서 실행하면, 폴더 안의 CLAUDE.md 라는 규칙서를
자동으로 읽습니다.

그래서 세 사람의 Claude 가 같은 규칙(금지사항·함정목록)을 따르게 됩니다.

비밀 열쇠 파일은 절대 깃에 올리지 않습니다

`.env`, `.dev.vars` 파일은 일부러 깃에 **없습니다**. 없으면 사장님께 따로 받으세요. 실수로 이 파일을 `git add` 해서 올리면, 결제·DB·AI 키가 통째로 노출돼 **전부 새로 발급받아야 합니다**. (실제로 과거에 한 번 겪어서 4단계에 걸쳐 열쇠를 전부 교체한 적이 있습니다.)

4. 누가 어떤 파일을 맡는가

충돌을 없애는 방법은 기술이 아니라 **약속**입니다. 이유는 파일 크기 때문입니다.

왜 파일을 나눠야 하나요

`index.html` 은 **1.6MB**, `admin.html` 은 **1.3MB** 짜리 거대한 한 덩어리 파일입니다. 이 파일을 두 사람이 동시에 고치면, 깃이 "어느 쪽이 맞는지" 판단하지 못해 **충돌 해결이 사실상 불가능**합니다. 잘못 풀면 남의 작업이 통째로 사라집니다.

4-1. 영역 분담표

영역	담당	파일
학생 학습 콘텐츠	직원 A·B	<code>public/student-game-*.html</code> (게임들) · <code>public/student-games.html</code> · <code>public/vocab.html</code> · <code>public/micro-quiz.html</code> · <code>public/review-quiz.html</code> · <code>public/warmup.html</code> · <code>public/ai-write.html</code> · <code>public/speech-coach*.html</code> · <code>public/suspect-mystery.html</code> · <code>src/api-games.ts</code> · <code>src/api-points.ts</code>
관리자·운영·정산	사장님	<code>public/admin.html</code> · <code>src/api-admin.ts</code> · <code>src/accounting-*.ts</code> · <code>src/api-payroll-auto.ts</code> · <code>src/org-settlement.ts</code>
공동 금지구역	사장님만	<code>public/index.html</code> (홈+화상수업) · <code>src/index.ts</code> (라우팅+로그인 관문) · <code>src/video-call-room.ts</code> · <code>src/signaling-room.ts</code> · <code>wrangler.toml</code> · <code>deploy.ps1</code> · <code>public/sw.js</code>

(모든 경로는 `cloudflare-deploy/` 아래입니다.)

4-2. 직원 A 와 B 는 어떻게 나누나

게임은 **파일이 하나씩 따로** 있어서 서로 안 겹칩니다. 그래서 "**파일 단위**"로 나누면 충돌이 0에 가깝습니다.

- 예) A = 단어장·복습퀴즈·웜업 / B = 게임들·발음코치·AI 영작
- 단, `api-games.ts` , `api-points.ts` 는 둘 다 쓰는 **공용 파일**입니다. 여기를 고칠 땐 **먼저 상대방에게 한마디** 하세요.
- 내 영역 밖 파일을 꼭 고쳐야 하면 **혼자 고치지 말고 사장님께 먼저 말씀**하세요.

5. 매일 하는 4단계

1 아침에 최신본 받기 — 커피보다 먼저

```
git checkout main
git pull
```

어제 남이 고쳐놓은 것을 내 컴퓨터로 가져옵니다.
이것만 지켜도 문제의 절반은 안 생깁니다.

2 내 작업방(브랜치) 만들기

```
git checkout -b 단어장-정렬수정
```

원본을 직접 건드리지 않고

나만의 복사본

에서 작업합니다. 여기서 뭘 망가뜨려도 남에게 영향이 없습니다.

이름 예시: **로그인-오류수정**, **게임-속도개선**. 한글도 됩니다.

3 다 고쳤으면 도장 찍고 올리기

```
git add -A
git commit -m "단어장 목록을 가나다순으로 정렬"
git push -u origin 단어장-정렬수정
```

`-m` 뒤 따옴표 안에는

무엇을 했는지 한글로

적습니다. 나중에 "이거 누가 왜 고쳤지?" 할 때 이 문장을 봅니다.

한 번 `-u` 로 올리고 나면, 그 다음부터는 `git push` 만 치면 됩니다.

4 사장님께 결재 올리기 (Pull Request)

GitHub 사이트에 들어가면

"Compare & pull request"

초록 버튼이 보입니다. 눌러서 제출하세요. 제목에 무엇을 고쳤는지, 본문에

왜 고쳤는지

와

어떻게 확인했는지

를 적습니다.

"내가 고친 게 홈페이지에 안 보여요" — 정상입니다

4단계까지는 **제안**일 뿐입니다. 사장님이 검사하고 합쳐야(6장), 그리고 배포돼야(8장) 화면에 나옵니다.

5-1. 커밋은 자주, 작게

하루 종일 고치고 저녁에 한 번 커밋하면, 문제가 생겼을 때 **어디서부터 잘못됐는지 찾을 수 없습니다**. "한 가지 일이 끝날 때마다" 커밋하세요. 커밋은 공짜입니다.

6. 합치기(merge) — 사장님이 검사하는 법

6-1. PR 화면에서 무엇을 보나

1. **Files changed** 탭 — 바뀐 줄이 초록(추가) / 빨강(삭제)으로 보입니다.
2. 바뀐 파일이 담당 영역 안인지 먼저 봅니다. 직원 PR 에 `index.html` 이나 `wrangler.toml` 이 들어 있으면 그 자리에서 되돌려 보내세요.
3. 비밀 열쇠(`.env` , 토큰 문자열)가 섞여 있지 않은지 확인합니다.

"파일 53개가 바뀌었다"고 나와도 놀라지 마세요

배포할 때마다 모든 HTML 맨 아래에 **배포 시각 도장**이 새로 찍힙니다.

```
<!-- BUILD:20260811145740 -->
```

그래서 배포 직후 `git pull` 하면 53개가 바뀐 것처럼 보이지만, 대부분 이 **1줄짜리 도장**뿐입니다. 진짜 변경만 보려면 도장을 빼고 봅니다:

```
git diff -I'BUILD:' main..작업브랜치
```

6-2. 충돌(conflict)이 났을 때

충돌 지점	어떻게 하나
<code><!-- BUILD:숫자 --></code> 줄	아무 쪽이나 골라도 됩니다. 어차피 다음 배포 때 새로 찍힙니다.
그 외 모든 곳	직원은 혼자 풀지 마세요. 사장님께 알리고 같이 봅니다. 잘못 풀면 남의 작업이 사라집니다.

6-3. 깃이 못 잡는 충돌 — "보이지 않는 충돌"

이게 제일 무섭습니다

두 사람이 같은 버그를 각자 다르게 고치면, 두 수정이 서로 다른 줄에 있기 때문에 깃은 **충돌이라고 알려주지 않고 조용히 둘 다 넣어버립니다**. 결과는 "고침이 두 번 적용된" 이상한 화면입니다. (실제로 겪었습니다.)

예방책은 하나뿐입니다 — 시작하기 전에 "나 이거 고칠게요" 라고 말하는 것. 원격이라 서로 뭘 하는지 안 보입니다. 아침에 한 줄씩 공유하세요.

6-4. 합치는 방법

검사가 끝나면 PR 화면에서 **Merge pull request** 버튼을 누릅니다. 그 순간 `main` 이 바뀌고, 자동 배포가 시작됩니다 (8장).

이 저장소는 무료 플랜이라 `main` 을 기술적으로 잠글 수 없습니다

즉 마음만 먹으면 누구나 `main` 에 바로 push 할 수 있습니다. 규칙으로만 막힙니다. 직원은 절대 `main` 에 직접 push 하지 마세요. (그 push 하나가 즉시 배포로 이어집니다.)

7. 검사 3종 — "고쳤습니다"만 말하면 안 됩니다

코드를 고쳤으면 **PR** 을 올리기 전에 내 컴퓨터에서 아래 3가지를 돌립니다. 그리고 PR 본문에 결과를 붙여 넣으세요.

검사 1. 문법 검사 (약 20초)

```
cd cloudflare-deploy
node node_modules/typescript/bin/tsc --noEmit
```

함정 — `npx tsc` 를 쓰면 안 됩니다

`npx` 는 조용히 실패해도 아무것도 안 찍습니다. 통과했을 때와 화면이 똑같습니다. "출력이 없으니 통과했다"고 착각하고 배포해서 사고가 난 적이 있습니다. 반드시 위 `node ...` 형태로 쓰세요.

검사 2. 회귀 하니스 (약 90초) — 가장 중요

```
# 반드시 «저장소 맨 위 폴더»에서 실행
node test-harness/run.mjs --fast
```

기준선: **PASS 114~115 / SKIP 20 / FAIL 0**. FAIL 이 하나라도 있으면 **PR** 을 올리지 마세요.

SKIP 20개는 실제 화상수업 화면이 필요한 검사라 자동으로 빠집니다 — 정상입니다.

함정 — 실행 위치

검사 1 에서 `cd cloudflare-deploy` 를 했으므로, **이어서 그대로 치면 실패합니다** (`MODULE_NOT_FOUND`). `cloudflare-deploy/test-harness/` 에도 같은 이름 폴더가 있지만 그 안엔 `run.mjs` 가 없습니다. 꼭 저장소 맨 위로 돌아가서 실행하세요 (`cd ..`).

이 하니스 138개 하나하나가 "예전에 이렇게 고장 났었다"는 사고 기록입니다. 그래서 이걸 통과하면 같은 사고가 다시 나지 않습니다.

검사 3. 실제 브라우저 확인

화면을 고쳤으면 반드시 눈으로 봅니다. 캡처를 PR 에 첨부하면 사장님이 훨씬 빨리 승인합니다.

보고 방식 (약속)

"고쳤습니다"라고만 하지 말고 **무엇으로 확인했는지** 같이 씁니다.

○ 좋은 예 : "단어장 정렬 수정. tsc 통과 / 하니스 PASS 114 FAIL 0 / 크롬에서 가나다순 확인(캡처 첨부)"

× 나쁜 예 : "수정 완료했습니다"

실패했으면 실패했다고, 건너뛴 게 있으면 건너뛰었다고 그대로 쓰세요. 확인 안 된 걸 "됐습니다"라고 하는 게 가장 큰 사고의 원인입니다.

8. 배포 — 길이 두 개 있고, 서로를 모릅니다

길 A. main 에 합치면 자동 배포 (권장)

PR 을 `main` 에 머지하는 순간, GitHub Actions 가 **알아서** 아래를 순서대로 합니다.

순서	하는 일	실패하면
1	필요한 프로그램 설치 (<code>npm ci</code>)	전부 "차단" 게이트입니다. 하나라도 실패하면 배포하지 않고 멈춥니다.
2	문법 검사 (tsc)	
3	회귀 하니스 138종	
4	HTML 안 JS 문법 전수 검사	
5	정적파일 캐시버전(<code>?v=</code>) 누락 검사	
6	Cloudflare 에 Worker 2개 배포 (기본 + production — 실서비스는 production 쪽입니다)	경고 표시
7	배포 후 접속 확인(헬스체크)	경고 표시

진행 상황은 GitHub 저장소 → **Actions** 탭에서 실시간으로 볼 수 있습니다. 초록 체크가 뜨면 성공입니다.

길 B. 사장님 PC 에서 직접 배포 (`deploy.ps1`)

```
powershell -File deploy.ps1
```

이 스크립트는 검사 → 도장찍기 → 커밋 → 배포 → 재확인까지 한 번에 합니다.

직원은 이 파일을 절대 실행하지 않습니다

이 스크립트는 내 컴퓨터의 `cloudflare-deploy/public/` 폴더를 통째로 업로드합니다. 즉 커밋하지 않은 미완성 파일까지 그대로 실서비스에 올라갑니다. 학생 29,000명이 즉시 그 화면을 보게 됩니다.

사장님도 실행 전 `git status` 확인은 필수입니다. 스크립트가 "이번에 올라갈 파일 목록"을 화면에 뿌려 주니, 모르는 파일이 보이면 **Ctrl+C** 로 즉시 중단하세요.

8-1. 두 길이 충돌하면 생기는 사고 (실제 사례)

"합쳤을 뿐인데 라이브가 3일 전으로 되돌아갔다"

사장님이 `deploy.ps1` 로 5번 배포했는데 그 내용이 `main` 에는 없었습니다. 그 상태에서 다른 PR 을 `main` 에 머지하자, 자동배포가 옛날 `main` 내용으로 라이브를 덮어썼습니다. 그 5번의 작업이 라이브에서 사라졌습니다.

예방책 : 로컬 배포를 했으면 그 브랜치를 반드시 곧 `main` 에 합치세요. 로컬 배포는 "임시", 진짜 정본은 언제나 `main` 이어야 합니다.

8-2. 배포 전 마지막 확인 한 가지

"지금 내 커밋이 이 브랜치의 맨 앞(tip)인가?" — 배포 직전에 확인합니다.

```
git log --oneline -3
```

과거에 "남의 미완성 파일을 피하려고" 내 커밋 시점에 고정해 배포했다가, 그 사이 남이 정상적으로 커밋한 11개 작업이 라이브에서 통째로 사라진 적이 있습니다.

9. test.mangoi.co.kr 에서 실제로 확인하기

주소를 헛갈리지 마세요

실서비스 주소는 `https://test.mangoi.co.kr` 입니다. 일부 옛 문서와 스크립트에 `mango-i.com` 이 적혀 있는데, 그 도메인은 등록조차 되어 있지 않습니다(접속 불가). 고객에게 나가는 링크에 그걸 쓰면 죽은 링크가 발송됩니다.

9-1. 1단계 — 캐시부터 지우세요 (99%가 여기서 착각합니다)

배포는 잘 됐는데 내 화면만 옛날 그대로인 경우가 대부분입니다. 브라우저가 옛 파일을 저장해 두기 때문입니다.

기기	확인 방법
PC	시크릿 창(Ctrl+Shift+N)으로 여는 게 가장 확실합니다. 또는 Ctrl+Shift+R (강력 새로 고침)
휴대폰	브라우저 설정 → 사이트 데이터 → <code>test.mangoi.co.kr</code> 데이터 완전 삭제 또는 시크릿/InPrivate 모드로 접속 (가장 확실)
앱으로 설치한 경우	홈 화면 아이콘을 길게 눌러 앱 삭제 후 재설치

9-2. 2단계 — "언제 배포된 버전인지" 확인

화면 소스 맨 아래에 있는 배포 도장을 봅니다.

```
# 배포 도장 확인 (캐시 우회를 위해 뒤에 아무 숫자나 붙임)
curl --compressed -s "https://test.mangoi.co.kr/?nocache=1" | findstr "BUILD:"
```

함정 — BUILD 도장만 믿으면 안 됩니다

이 도장은 `deploy.ps1` (길 B)로 배포할 때만 새로 찍힙니다. **GitHub Actions** 자동배포(길 A)는 도장을 새로 찍지 않습니다.

그래서 "도장이 그대로다 = 배포가 안 됐다" 는 틀린 판단입니다. 내가 고친 그 글자·그 기능이 실제로 보이는지로 판단하세요. 그게 유일하게 확실한 방법입니다.

9-3. 3단계 — 실제 눈으로 확인 (필수)

- 시크릿 창으로 `https://test.mangoi.co.kr` 접속
- 내가 고친 그 화면으로 직접 이동해서 고친 부분이 보이는지 확인
- 휴대폰에서도 한 번 더 확인 (화면 크기 때문에 PC에선 멀쩡한데 폰에서 깨지는 일이 흔합니다)

4. 고친 기능을 실제로 눌러서 동작하는지 확인 (보이는 것 ≠ 작동하는 것)

확인 순서 요약

① Actions 탭 초록 체크 → ② 시크릿 창으로 접속 → ③ 내가 고친 부분이 보이는지 → ④ 눌러서 동작하는지 → ⑤ 휴대폰에서 한 번 더

10. 사고가 났을 때 (되돌리기)

10-1. 아직 커밋 안 했는데 파일을 망쳤다

```
git checkout .
```

마지막 커밋 상태로 되돌아갑니다. 단, 그 이후 작업한 건 전부 사라집니다. 아까우면 실행 전에 물어보세요.

10-2. 이미 합쳤는데 잘못됐다

```
git revert <커밋번호>
```

"그 변경을 취소하는 새 커밋"을 만듭니다. 기록은 남기면서 되돌리는, 가장 안전한 방법입니다.

10-3. 라이브가 깨졌다 — 가장 급한 상황 (사장님만)

```
cd cloudflare-deploy
npx wrangler rollback
npx wrangler rollback --env production # 실서비스는 이쪽이 진짜
```

직전 배포 버전으로 즉시 되돌립니다. 먼저 되돌리고, 원인은 그 다음에 찾으세요.

연락 순서

라이브가 이상하면 혼자 해결하려 하지 말고 즉시 사장님께 연락하세요. 학생 29,000명이 쓰는 서비스입니다. 억지로 고치려다 더 크게 만드는 경우가 가장 많습니다.

11. Claude 를 셋이 같이 쓰는 법

11-1. 무엇이 공유되고, 무엇이 안 되나

항목	공유?	설명
CLAUDE.md (규칙서)	공유 ○	저장소에 들어 있어서 세 사람의 Claude 가 전부 같은 규칙을 읽습니다. 새 함정을 발견하면 여기에 적고 PR 에 포함하세요 — 그러면 다른 두 사람의 Claude 도 그걸 알게 됩니다.
Claude 의 "기억"	공유 ✕	각자 PC 안에만 있습니다. 내 Claude 가 아는 걸 남의 Claude 는 모릅니다. 중요한 건 사람이 직접 말해 주거나 CLAUDE.md 에 적어야 합니다.
대화 내용	공유 ✕	서로 보이지 않습니다.

11-2. Claude 에게 일을 시킬 때의 3가지 습관

1 "먼저 git pull 하고 시작해줘"

Claude 는 내 폴더 상태만 봅니다. 옛날 상태에서 작업하면 나중에 충돌합니다.

2 "이건 내 담당 파일이야" 를 알려주기

Claude 는 도움이 되려고

관련 있어 보이는 다른 파일까지 고치려

합니다. 금지구역(`index.html` 등)은 열지 말라고 미리 못 박으세요.

3 "무엇으로 확인했는지 같이 보고해줘"

검사 3종(7장)을 돌리고 결과를 함께 말하게 하세요. "고쳤습니다"만 받으면 확인이 안 된 것입니다.

세 사람의 Claude 가 같은 폴더를 동시에 만지면 안 됩니다

실제로 겪은 사고입니다. 한 세션이 작업하는 도중 다른 세션이 폴더를 되돌려서 **미커밋 작업이 통째로 사라졌습니다**. 커밋은 자주 하세요. 커밋하지 않은 작업은 언제든지 사라질 수 있습니다.

12. 절대 하지 말 것

금지	이유
DB 에 DELETE / UPDATE / DROP	개발용 DB 와 운영 DB 가 같습니다. 테스트용이 따로 없습니다. 실제 학생 29,000명의 데이터입니다. 확인은 SELECT 만. 변경이 필요하다면 반드시 사람에게 먼저 말하세요.
직원이 deploy.ps1 실행	내 컴퓨터의 미완성 파일이 그대로 실서비스에 올라갑니다.
main 에 직접 push	검사 없이 곧바로 자동 배포로 이어집니다. 무료 플랜이라 기술적으로 못 막습니다 — 규칙으로만 지켜집니다.
.env / .dev.vars 커밋	결제·DB·AI 열쇠가 통째로 노출됩니다. 전부 재발급해야 합니다.
남의 담당 파일 수정	충돌 해결이 사실상 불가능합니다. 꼭 필요하다면 먼저 말하고 하세요.
"없길래 다시 넣었어요"	일부러 없앤 기능들이 있습니다 (관리자 hover 카드, 화상수업 원업 버튼, 관리자 서비스워커 등). 없는 데는 이유가 있습니다. CLAUDE.md 를 먼저 읽으세요.
JS/CSS 고치고 ?v= 안 올리기	"배포는 됐는데 기존 사용자에게 옛 파일" 상태가 됩니다. 화면은 새것, 스크립트는 옛것이라 원인 찾기가 특히 어렵습니다. (다행히 검사 게이트가 잡아 줍니다 — 그래서 검사를 건너뛰면 안 됩니다.)

13. 용어사전 — 서류함으로 비유하면

깃 용어	실제로 무슨 뜻이냐면
<code>git pull</code>	서류함에서 최신 서류 꺼내오기
브랜치 (작업방)	원본 말고 복사본에 연필로 수정하기
<code>git add</code>	제출할 서류를 봉투에 담기
<code>git commit</code>	"여기까지 고쳤음" 하고 도장 찍기 (아직 내 책상 위)
<code>git push</code>	도장 찍은 봉투를 서류함에 제출하기
Pull Request (PR)	사장님께 "원본에 반영해 주세요" 결재 올리기
머지 (merge)	결재가 나서 원본에 실제로 합쳐지는 것
충돌 (conflict)	두 사람이 같은 줄을 고쳐서 깃이 판단을 못 하는 상태
배포 (deploy)	합쳐진 내용을 진짜 홈페이지에 올리는 것 (사장님만)
롤백 (rollback)	배포를 직전 버전으로 되돌리는 것
워크트리 (worktree)	같은 저장소를 쓰는 별도 작업 폴더 (사장님 전용 고급 기능)

14. 요약 카드 — 책상에 붙여두세요

직원용 — 매일 이 순서

```
아침      : git checkout main
           : git pull                      ← 커피보다 먼저

작업 시작 : git checkout -b 작업이름

작업 중   : (자주) git add -A
           : git commit -m "무엇을 했는지"

PR 전 검사 : cd cloudflare-deploy
           : node node_modules/typescript/bin/tsc --noEmit
           : cd ..
           : node test-harness/run.mjs --fast      ← PASS 114 / FAIL 0
           : 브라우저에서 눈으로 확인

제출      : git push -u origin 작업이름
           : GitHub 에서 "Compare & pull request"
           : PR 본문에 «무엇으로 확인했는지» 쓰기

절대 금지 : deploy.ps1 실행 / main 에 직접 push / DB 변경 / .env 커밋
```

사장님용 — 합치고 배포하고 확인하기

1. PR 검사 : 바뀐 파일이 담당 영역 안인가? 비밀키 섞였나?
git diff -I'BUILD:' main..작업브랜치 ← 도장 빼고 진짜 변경만
2. 합치기 : GitHub 에서 Merge pull request
→ Actions 가 검사 5중 후 자동 배포 (초록 체크 확인)

또는 : powershell -File deploy.ps1 ← 로컬 배포 (git status 먼저!)
△ 로컬 배포했으면 그 브랜치를 «반드시» main 에 합칠 것
3. 확인 : 시크릿 창으로 https://test.mangoi.co.kr
내가 고친 부분이 «실제로» 보이는가 → 눌러서 동작하는가
휴대폰에서도 한 번 더
△ BUILD 도장은 자동배포 때 안 바뀜 – 내용으로 판단할 것
4. 사고 시 : cd cloudflare-deploy
npx wrangler rollback --env production ← 먼저 되돌리고 원인은 나중에

마지막으로 — 이 세 가지만 지켜도 사고의 90%는 안 납니다

① 아침에 `git pull` · ② 내 담당 파일만 만지기 · ③ 확인 안 된 것을 "됐습니다"라고 말하지 않기

망고아이 3인 공동개발 가이드 · 2026-08-11 작성

상세 규칙서 : 저장소의 `CLAUDE.md` · `MAINTENANCE.md` · `docs/TROUBLESHOOTING.md`

모르는 것은 혼자 판단하지 말고 물어보세요.